

Data Integration for the Semantic Web with Full Preferences

Olivier Curé

Université Paris Est, Institut Gaspard Monge,
77454 Marne la Vallée Cedex 2, France
olivier.cure@univ-paris-est.fr

ABSTRACT

This paper presents a tool that enables the integration of data stored in relational databases into Semantic Web compliant knowledge bases. The resulting knowledge bases are represented using Description Logics and can thus be translated into the Web Ontology language (OWL). Our approach tackles the impedance mismatch problem which is due to the storage of data in (relational) databases and objects in the knowledge bases. This problem is addressed with a mapping language that allows to specify how to transform data into objects. Another issue undertaken by our solution is related to inconsistencies emerging when contradicting values can be integrated into a given object. In order to deal with these inconsistencies, we enable users to set preferences over mapping views and their attributes. This approach provides a fine grained solution to design consistent knowledge bases.

Categories and Subject Descriptors

H.4 [Information Systems Applications]: Miscellaneous;
I.2.4 [Knowledge Representation Formalisms and Methods]: Representation languages

General Terms

Algorithms, Design

Keywords

Schema Mapping, Data Integration, Knowledge Base

1. INTRODUCTION

Integration and exchange of data enable to combine data residing at different sources by providing a uniform and reconciled target. Most research work in these fields have tackled the domain of (relational) databases for both sources and targets. Recently, the emergence of the Semantic Web has motivated research in these two fields where Description Logics (DL) [2] knowledge bases are considered as models for sources as well as targets.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

ONISW'08, October 30, 2008, Napa Valley, California, USA.
Copyright 2008 ACM 978-1-60558-255-9/08/10 ...\$5.00.

In this paper, we consider the situation where the sources are relational databases and the target is a DLs knowledge base. This approach is mainly motivated from the fact that designing ontologies from scratch is a complex, error-prone and time-consuming task. Hence we consider that available structured background information on a given domain can be helpful sources in designing an ontology of this domain.

Relational databases are obvious candidates for such background information as they are currently considered the most popular model to store information. But there are other reasons to consider them as valuable sources in helping to design ontologies and instantiate knowledge bases. One of these reasons emerges from the structural similarity between DLs and databases, i.e. they both propose intensional, i.e. schema for databases and TBox for DL, and extensional components, i.e. database instance and ABox for DL. In this paper, we are interested in both of these components as: (i) the intensional part of a database can help us to design a practical ontology, via a the declaration of a mapping, (ii) the extensional aspect enables to retrieve data from the sources, using the mappings, and integrate them in the knowledge base. Considering the intensional components, the authors of [15] emphasize their similarities as they are both interpreted according to standard first-order logic semantics. Considering this point of view, DL knowledge bases can be considered as expressive but decidable database schema languages. In contrast, database schemata often do not provide explicit semantics for their data. Thus, in order to benefit from DLs expressiveness and associated schema reasoning procedures, it is necessary to enrich the data integration target with DL axioms. We consider that this enrichment step is a semi-automatic process and requires end-users to provide definitions to their concepts at the time of defining mappings between elements of the sources and the target. We consider that the goal of this approach is to create valuable knowledge bases which are being used in data-centric applications requiring inferences, e.g. [4] [5].

Thus, in this paper, we do not consider query answering and constraint checking discrepancies between these two models and assume that in the knowledge base, they are being dealt with the standard open-world assumption.

A first important issue we can consider in the setting of this integration solution is the impedance mismatch. This problem is due to the fact that databases store data while knowledge bases represent objects. We handle this issue by providing a mapping language that transforms data retrieved from tuples of the database to objects of the knowledge base.

Another problem one can face when integrating several sources is to obtain an inconsistent knowledge base. These inconsistencies can arise from uncertain mappings and overlapping sources. The solution proposed to tackle this issue is based on setting preferences over mapping assertions. Thus in situation of overlapping sources end-users are able to define which source is more reliable to another. But this approach is not sufficient to obtain fine-grained knowledge bases as it forces data transformed for an object to come from a single source. In practice, this is not a realistic and desirable approach.

For instance, let consider two sources S and R with respective schema $\langle s_1, \dots, s_n \rangle$ and $\langle r_1, \dots, r_n \rangle$ and equivalent domains for attributes s_i and r_i with $1 \leq i \leq n$. For these sources, end-users may consider that s_1 is more reliable than r_1 but that s_2 is less reliable than r_2 . In order to represent these source beliefs in the knowledge base, via the execution of mapping assertions, it is required to support preferences on the attributes of the mapping assertions.

This paper is organized as follows: in Section 2, we provide some basic notions and present the integration system, giving details about our mapping syntax and semantics. Section 3 highlights the preference solutions that have been implemented in our system. They correspond to preferences over views of the mapping, a solution already presented in [6], and an its extension with preferences over attributes of the views. The conjunction of these two preferences is denoted Full preferences. In Section 4 we present an algorithm enabling the instantiation of a knowledge base from sources given a mapping with full-preferences. Section 5 presents an implementation of this system and provides a preliminary evaluation. Section 6 presents some related work and we conclude in Section 7.

2. MAPPING AND THE IMPEDANCE MISMATCH PROBLEM

Before introducing the impedance mismatch problem, we must provide some basic notions about the data models of sources and targets of our data integration system.

2.1 Basic notions

The sources correspond to relational databases and our data integration approach uses their relational schemata. A relational schema $\mathcal{R}$ contains a finite set of relations. Each relation in $\mathcal{R}$ contains a finite set of attributes and is denoted by $R = \langle r_1, \dots, r_n \rangle$ where n is the arity of the relation. An instance I_R of R is a finite set of tuples, where each tuple associates a value to each attribute in the schema.

The targets designed and instantiated with our system correspond to DL knowledge bases. This family of knowledge representation formalism allows to represent and reason over domain knowledge in a formally and well-understood way. Central DLs notions are concepts (unary predicates) and relationships (binary predicates), also called roles or properties. A key notion in DLs is the separation of the terminological (or intensional) knowledge, called a TBox, to the assertional (or extensional) knowledge, called a ABox. When considered together, a TBox and a ABox represent a knowledge base denoted $\mathcal{K} = \langle \text{TBox}, \text{ABox} \rangle$. A specificity of DLs is to natively integrate some reasoning procedures [2]. Standard reasoning procedures for the TBox are concept satisfiability and concept subsumption which respec-

tively check whether a concept can have instances in models of the TBox, and whether one concept is more general than another concept with respect to models of the TBox. A model of an TBox is a satisfied interpretation of the TBox.

The impedance mismatch problem is related to the fact that relational databases store data while knowledge bases represent objects and their relationships. Our solution handles this issue by means of a declarative mapping language that enables the transformation of data into objects. In the next subsections, we present the syntax and semantics of this mapping system.

2.2 Syntax

The mapping approach we have adopted in our system is GAV (Global-As-View) with sound sources and requires that the target schema is expressed in terms of queries over the sources [11]. Intuitively, an end-user maps a DL concept, respectively role, to a query and in a second step, she maps all distinguished variables of the query to previously defined datatype properties, respectively concepts of the ontology. We now formalize this intuition of the mapping system.

Definition 1: Our data integration system can be defined as $\mathcal{DI} = (\mathcal{S}, \mathcal{K}, \mathcal{M})$, where:

- $\mathcal{S}$ is a set of relational database source schemas, $\{S_1, \dots, S_n\}$ that we assume locally satisfy their set of integrity constraints.
- $\mathcal{K}$ is the (target) ontology schema formalized in a DL knowledge base.
- $\mathcal{M}$ is the mapping between $\mathcal{S}$ and $\mathcal{K}$. This mapping is represented as a set of GAV assertions in which views, i.e. queries, expressed over elements of $\mathcal{S}$ are put in correspondence to elements of the TBox of $\mathcal{K}$.

It is important to note that this approach enables two different ontological engineering approaches:

- creation of a TBox from scratch, i.e. $\mathcal{K}$ is empty at the start of the design process. The end-users is supposed to create datatype properties which will be later mapped to attributes of the mapping queries. It is also possible to define DL concepts and object properties and to use them to design hierarchies.
- start from an existing TBox and enrich it with new concepts and/or generate concept/role assertions from loaded relational sources via the execution of mapping assertions.

We can now present the form of mapping assertions that have been adopted in our system.

Definition 2: The mapping assertions in $\mathcal{M}$ for $\mathcal{DI}$ take the following form:

$$\forall \bar{x}(\phi(\bar{x}) \rightarrow \exists \bar{y}\psi(\bar{x}, \bar{y}))$$

where ϕ denotes either a *Concept* or an *Object property* with arity of $\bar{x}$ attribute mappings and ψ denotes a conjunctive query over source relations. We denote as ϕ the premise of a mapping rule and ψ its conclusion. The two forms of ϕ , therefore named DI-premises, correspond to:

- *Concept* where tuples retrieved from the conclusion serve to create instances of this concept.

- *Object property* where tuples retrieved from the conclusion serve to relate existing ABox instances.

Finally, we present the attribute mappings that are associated to each form of DI-premises:

- if the query attribute is a primary key in its source relation, then the end-user may select this value to be an object identifier in the knowledge base. This definition is supported by a special property defined in an associated mapping ontology, namely “mapping:id”. This property creates a linked list of objects where each object has a “hasIdValue” property whose value is retrieved from the query. This approach enables to relate source relations with compound primary keys to objects in the knowledge base.
- if the query attribute is not a primary key, the end-user maps it to a datatype property.

In DL, roles correspond to binary predicates where a first component relates to the domain and a second to the range. The same approach applies to *Object property* DI-premises. As previously explained, our system supports compound keys and thus a non-empty set of distinguished variables may identify the domain and range. It is obvious in our system that the distinguished variables of an *Object property* mapping assertion must correspond to attributes that have been previously, in a *Concept* DI-premise mapping, mapped to the *mapping:id* property. The main idea of the system is to define the set of distinguished variables that correspond to the domain, respectively the range and based on the values retrieved from the execution of this query, identify the knowledge base individuals and relate them via the object property.

Example 1 Consider the data integration system $\mathcal{DI}_0$ which we define by $(\mathcal{S}_0, \mathcal{K}_0, \mathcal{M}_0)$, where the source $\mathcal{S}_0$ contains two relations:

- *employee* with 4 attributes: an employee identifier (id), employee name (name), first name (fname) and salary (salary).
- *manage* which relates employees (idE attribute) to their managers (idM).

Some tuples of these relations are displayed in Figure 1.

We consider that $\mathcal{K}_0$ contains the following datatype properties: *hasName*, *hasFirstName* and *hasSalary* which have *Person* as domain and respectively *string*, *string* and *integer* values for their range.

According to the above description of the source, we propose the following mapping assertions for $\mathcal{M}_0$ which define a concept, i.e. *Employee*, and an object property, i.e. *hasManager*:

$Employee \rightsquigarrow \{w, x, y, z | employee(w, x, y, z)\}$. The distinguished variables of the view are then mapped to some datatype properties of the target ontology or to our mapping ontology, i.e. $w \rightsquigarrow mapping:id$, $x \rightsquigarrow hasName$, $y \rightsquigarrow hasFirstName$, $z \rightsquigarrow hasSalary$
 $hasManager \rightsquigarrow \{x, y | manage(x, y)\}$ with $x \rightsquigarrow Employee$ and $y \rightsquigarrow Employee$

2.3 Semantics

First, it is important to not that a standard first-order semantics can interpret DL TBoxes and relational schemata

employee Relation

manage Relation

id	name	fname	salary	idM	idE
1	aE	faE	10	8	1
2	bE	fbE	12	8	2
3	cE	fcE	13	9	3
8	hE	fhE	15		
9	iE	fiE	15		
10	jE	fjE	17		

Figure 1: Employee relation in Source 1

because they both distinguish legal structures, i.e. those that satisfy all axioms, named models in DL and database instances in relational databases, from illegal ones, i.e. structures that violate some of them. Thus we can use a first-order semantics with the domain of interpretation being a fixed denumerable set of elements Δ , and every such element is denoted uniquely by a constant symbol in Γ . In this setting, constants in Γ act as *standard names* [12].

In order to specify the semantics of $\mathcal{DI}$, we first have to consider a set of data at the sources, and we need to specify which are the data that satisfy the target schema with respect to such data at the sources. We call $\mathcal{C}$ a *source model* for $\mathcal{DI}$.

Starting from this specification of a *source model*, we can define the information content of the target $\mathcal{K}$. From now on, any interpretation over Δ of the symbols used in $\mathcal{K}$ is called a *target interpretation* for $\mathcal{DI}$.

Definition 3: Let $\mathcal{DI} = \langle \mathcal{S}, \mathcal{K}, \mathcal{M} \rangle$ be our data integration system, $\mathcal{C}$ be a *source model* for $\mathcal{DI}$, a *target interpretation* $\mathcal{A}$ for $\mathcal{DI}$ is a model for $\mathcal{DI}$ with respect to $\mathcal{C}$ if the following holds:

1. the ABox $\mathcal{A}$ is consistent with respect to the TBox $\mathcal{K}$.
2. $\mathcal{A}$ satisfies the mapping $\mathcal{M}$ with respect to $\mathcal{C}$.

The first condition in Definition 3 can be handled with the DL standard ABox consistency reasoning solution. For the kind of DL we are using in our implementation, namely *SHIF(D)* and *SHOIN(D)*, corresponding respectively to OWL Lite and OWL DL [7], this reasoning problem is decidable but the complexities are ExpTime-complete [17] and NExpTime-complete [16] respectively.

We also need to consider the constraints that need to be satisfied by $\mathcal{K}$. To do so we need to adopt a database-like constraint approach in the DL knowledge base. This approach is satisfied by the adoption of the Unique Name Assumption (UNA), i.e. requiring each object instance to be interpreted as a different individual. After this constraint checking step, UNA is relaxed in order to free DL knowledge bases from this restrictive aspect.

The introduction of UNA is supported by the mapping ontology *mapping:id* property which introduces the *mapping:idSequence* property. This property is considered as an object property and needs to satisfy the following axioms:

- (1) $\forall x, y_1, y_2 (\neg mapping : idSequence(x, y_1) \vee \neg mapping : idSequence(x, y_2) \vee y_1 = y_2)$
- (2) $\forall x, y_1, y_2 (\neg mapping : idSequence(y_1, x) \vee \neg mapping : idSequence(y_2, x) \vee y_1 = y_2)$

These constraints state that the domain of an object property is identified by a single range (1) and that a range identifies a single domain (2). In the implementation of our system, the *mapping:idSequence* property is defined in terms of cardinality constraints of OWL properties, that is *owl:functionalProperty*, i.e. equivalent to (1) and *owl:inverseFunctionalProperty*, i.e. equivalent to (2). Using this approach, we are still able to design ontologies in the decidable fragment of OWL, namely OWL DL.

The notion of $\mathcal{A}$ satisfying the mapping $\mathcal{M}$ with respect to $\mathcal{C}$ depends on the interpretation of the mapping assertions. In the context of our solution, we consider the GAV mappings to be sound, i.e. data that it is possible to retrieve from the sources through the mapping are assumed to be a subset of the intended data of the corresponding target schema [11]. In this case, there may be more than one legal knowledge base that satisfies the mapping $\mathcal{M}$ with respect to $\mathcal{C}$.

It is generally considered that DL knowledge bases can be understood as incomplete databases [13]. And there is a general agreement that in the context of incomplete databases, the 'correct' answers are the *certain answers*, that is, answers that occur in the intersection of all 'possible' databases. We adopt this notion for query answering in our system and use the results that the evaluation of conjunctive queries on an arbitrarily chosen universal solution gives precisely the set of certain answers and that universal solution are the only solutions that have this property [10]. In this same paper, the authors showed that in the context of a GLAV (Global-Local-As-View) mapping and tuple-generating (TGD) and equality-generating dependencies (EGD) on the target, the universal solution could be computed, if a solution exist, in polynomial time with the Chase [1]. In our setting, the dependencies we consider are functional dependencies, a form of EGD, and the source to target dependencies take the form of a GAV mapping, i.e. less expressive than GLAV. Thus our approach to compute the universal solution employs a chase-like approach and computes a universal solution, if a solution exist, in polynomial time. Fig.2 displays an extract of the universal solution computed in Example 1.

3. PREFERENCE-ENABLED SCHEMA MAPPINGS

Our data integration solution aims to integrate several relational database sources into an OWL knowledge base. A situation frequently encountered in integration solutions is the execution of uncertain mappings. Such mappings are often generated from overlapping sources where data are possibly not up-to-date and unreliable. In such a situation, one needs to give special attention to instantiate the most reliable, up-to-date and consistent knowledge base. In order to guarantee the consistency of the processed target when several mapping assertions are defined for a given DI-premise, end-users can set preferences, i.e. confidence values between 0 and 1, over the mapping rules. In [6], we presented a solution that enables to define preferences over conjunctive queries of mapping assertions, named *R-preferences*. This approach considers that data retrieved from a source all have the same reliability value.

Definition 4: Let $\mathcal{DI} = \langle \mathcal{S}, \mathcal{K}, \mathcal{M} \rangle$ be our data integration system with preference assertions. We consider the sit-

uation where an element of $\mathcal{K}$ (i.e. *Concept* or *Object property*) is defined with several mapping assertions, denoted ma_i over sources of $\mathcal{S}$. In order to avoid inconsistencies, the end-user defines a total order relation, denoted $>_R$, on the mapping assertions of this element. For two mapping assertions ma_1 and ma_2 , if $ma_1 >_R ma_2$ then

- all objects retrieved from a mapping assertion with no counter parts (i.e. objects identified by the same identifying values) in other mapping assertions are created in the ABox.
- all objects that could be retrieved from both mapping assertions, are created exclusively from values of ma_1 .

Example 2: Consider an integration system $\mathcal{DI}_1 = \langle \mathcal{S}_1, \mathcal{K}_1, \mathcal{M}_1 \rangle$ where the source $\mathcal{S}_1$ regroups 3 distinct relational databases which respectively contain data about employees, persons and citizens. The relation *employee* is the same as in Figure 1 and the other relations are displayed in Figure 3. These schemas have a common pattern with attributes relating to: identification, name, first name and salary.

Before the execution of $\mathcal{M}_1$, the knowledge base $\mathcal{K}_1$ is similar to $\mathcal{K}_0$ in Example 1, i.e. with the following datatype properties: *hasName*, *hasFirstName* and *hasSalary*. According to the above description of the source, we propose the following mapping assertions of $\mathcal{M}_1$:

$ma_1 : Employee \rightsquigarrow \{w, x, y, z | employee(w, x, y, z)\}$
with $w \rightsquigarrow mapping : id, x \rightsquigarrow hasName, y \rightsquigarrow hasFirstName, z \rightsquigarrow hasSalary$
 $ma_2 : Employee \rightsquigarrow \{w, x, y, z | person(w, x, y, z)\}$
with $w \rightsquigarrow mapping : id, x \rightsquigarrow hasName, y \rightsquigarrow hasFirstName, z \rightsquigarrow hasSalary$
 $ma_3 : Employee \rightsquigarrow \{w, x, y, z | citizen(w, x, y, z)\}$
with $w \rightsquigarrow mapping : id, x \rightsquigarrow hasName, y \rightsquigarrow hasFirstName, z \rightsquigarrow hasSalary$

On the *Employee* concept, the end-user has the ability to define the following total (reliability) order on the sources, e.g. $ma_1 >_R ma_2 >_R ma_3$. In this case, asking for employee names in the ABox returns the set: $\{aE, bE, cE, dC, eP, fP, gP, hE, iE, jE\}$.

This example emphasizes that with this approach, it is not possible to compose objects from different sources. That is all values of an object come from a single source. In order to propose a fine-grained composition of objects, we enriched our system with a preference setting over attribute of the views. They are denoted *A-preferences* and are set on the distinguished variables of mapping assertions. They enable to create fine-grained objects from source tuples. With this approach, one can express that for a given object, the system prefers the value of an attribute stored in one source to another attribute stored in another source. Thus the system is able to compose an object via retrieving preferred values from different sources.

In our extended mappings, both preferences can coexist within a mapping and we call *Full-preferences* the union of *R-preferences* and *A-preferences*. In order to ease the definition of mapping assertions, we have implemented a Graphical User Interface (GUI) that enables to set the *R-preference* of a mapping assertions as the default value for all *A-preferences* of this same mapping assertion. The end-user can later overrule all the *A-preferences* she wants.

Definition 5: Let $\mathcal{DI} = \langle \mathcal{S}, \mathcal{K}, \mathcal{M} \rangle$ be our data integration system with *Full-preferences* over mapping asser-

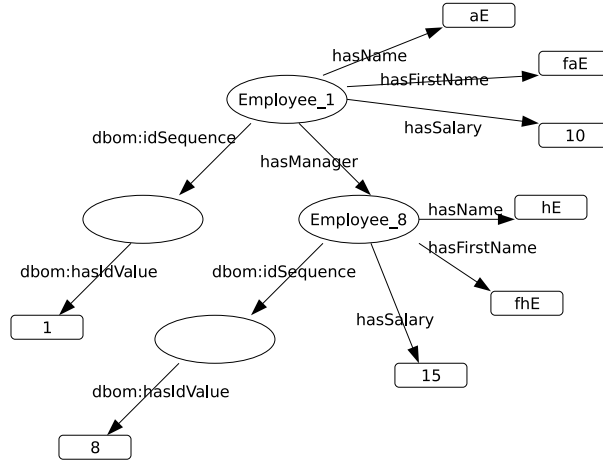


Figure 2: Extract of the graph generated for Example 1

tions. We consider the situation where an element of $\mathcal{K}$ (i.e. *Concept* or *Object property*) is defined with several mapping assertions, denoted ma_i over sources of $\mathcal{S}$. In order to avoid inconsistencies and to compose fine-grained knowledge bases, the end-user defines a total order relation, denoted $>_A$, on the distinguished variables of mapping assertions of this element. For two mapping assertions ma_1 and ma_2 and respective attributes set (a_1, b_1, c_1) and (a_2, b_2, c_2) where c_i correspond to identifiers (mapped to $dbom:id$), if $ma_{1,a_1} >_A ma_{2,a_2}$ and $ma_{2,b_2} >_A ma_{1,b_1}$ then

- all objects retrieved from a mapping assertion with no counter parts in other mapping assertions are created in the ABox.
- all objects that could be retrieved from both mapping assertions, are composed by relating a_1 from ma_1 and b_2 from ma_2 to respective datatype properties.

Example 3: We now enrich $\mathcal{DI}_1$ from Example 2 with *A-preferences*. The mapping $\mathcal{M}_1$ is extended with: $ma_{1,x} >_A ma_{2,x} >_A ma_{3,x}$

$ma_{2,y} >_A ma_{1,y} >_A ma_{3,y}$

$ma_{3,z} >_A ma_{2,z} >_A ma_{1,z}$

With these *A-preferences* the end-user consider that for the name attribute, the *employee* relation is more reliable than *person* relation which is also more reliable than the *citizen* relation. Asking for the values of the *Employee* instance identified by value 1, returns $(aE, faP, 29)$ for respectively the *hasName*, *hasFirstName* and *hasSalary* properties.

4. FULL-PREFERENCE ALGORITHMS

In this section, we present the algorithms that enable to retrieve data from tuples of the sources with respect to *Full-preferences* over mapping assertions. Let S be the set of sources involved in a mapping and A the set of attributes in a mapping assertion query. In A , we distinguish between attributes which are mapped to the *mapping:id* property, denoted A_{id} and the other attributes A_o such that $A = A_{id} \cup A_o$ and $A_{id} \cap A_o = \emptyset$. We denote the arity of a set X with $\|X\|$, e.g. arity of S is denoted $\|S\|$.

In the context of *Full-preferences*, the complexity of computing all solutions for a set S of sources and A of attributes is related to the number of possible permutations when the

order of values matters and a value can be selected several times in a rearrangement. The number of such permutations is: $\|S\|^{\|A\|}$. Although still proposing an intractable solution, we reduce the complexity of the problem to $2^{\|S\|}$ in the worst case and propose heuristics which makes the issue of computing *Full-preferences* over practical integration cases relatively efficient. The first algorithm searches for attribute permutations with respect to existing overlapping of the sources.

Algorithm 1. existing attribute permutations algorithm

Input : mapping $M \subseteq \mathcal{M}$ for a DI-premise

1. S = the set of sources involved in M
2. FOR EACH source s_i in S DO
3. store attributes in A_{id} in a list L_i
4. END DO
5. $set = getPowerSet(\emptyset, M)$
6. WHILE $\bigcup_i L_i \neq \emptyset$ AND $set \neq \emptyset$
7. $\alpha = \bigcap_{i \in set} L_i$
8. $generateQuery(\alpha, set, M)$
9. FOR EACH L_i in set
10. $L_i = L_i - \alpha$
11. END DO
12. $set = getPowerSet(set, M);$
13. END WHILE

In Algorithm 1, the *getPowerSet* method (line 5) returns a set of all subsets of S . Subsets are returned in descending order on their size and the emptyset terminates the algorithm. A heuristic first returns the singleton sets and then it analyzes if *A-preferences* do not overrule *R-preferences* such that the emptyset can be returned more efficiently, without considering all subsets. Otherwise other heuristics return sets composed of sources with the highest *R-preferences* first. The second method being called in Algorithm 1 is

person Relation				citizen Relation			
<u>idp</u>	<u>namep</u>	<u>fnamep</u>	<u>salaryp</u>	<u>idC</u>	<u>nameC</u>	<u>fnameC</u>	<u>salaryC</u>
1	aP	faP	30	1	aC	faC	29
3	cP	fcP	30	2	bC	fbC	24
5	eP	feP	34	3	cC	fcC	26
6	fP	ffP	37	4	dC	fdC	21
7	gP	fgP	37	6	fC	ffC	23
10	jP	fjP	37	7	gC	fgC	24
				9	hC	fhC	24

Figure 3: Person and Citizens source relations

the *generateQuery* method (line 8). This method rewrites the queries in M associated to sources stored in *set*. In order to improve our system’s performance when instantiating the ABox, we use parameterized queries which are compiled once and then re-used with different parameter values without recompilation. The parameter we use for the rewritten queries correspond to the values contained in α .

5. DBOM IMPLEMENTATION AND EVALUATION

The data integration system presented in this paper has been implemented in the DBOM (DataBase Ontology Mapping) system. This tool takes the form of a Protégé plug-in which enables interactions with all existing tabs of this knowledge and ontology editor. A screenshot of the DBOM system is proposed on Figure 4. On the top of the figure, we can see the standard Protégé tabs (Metadata, OWLClasses, Properties, Individuals) and the DBOM tab which has been added in our configuration. We can distinguish 5 different numbered areas in Figure 4 and we now detail each of them: (1) enables to load/remove relational databases, load previously defined schema mappings, save the current mapping and generate an ABox via the execution of the mapping assertions. (2) displays the schema of a loaded relational database. (3) enables to define elements of the ontology : concepts and object properties. In Figure 4, we display an example of a *Concept* DI-premise. The type of the instances is a concept named *HomeopathyDrug* which has been declared to be a subconcept of the *Drug* concept, previously defined in Protégé OWLClasses tab. (4) this panel enables to define the conclusion of our mapping rule. A conclusion takes the form of a SQL query and the end-user is not limited in the number of queries she can add for a given ontology element, i.e. the “add queries” button. Fig. 4 displays a query on Source 1 and the *R-preference* associated to this query is 54%. (5) this last panel enables to define the mapping associated to the attributes of the SELECT clause of the query. Each attribute is mapped to a previously defined datatype attribute of the ontology or to the DBOM role which defines an identifier for this object, i.e. *dbom:id*. The *R-preference* value is set as default for all *A-preferences* of the mapping. In our screenshot, we can see that the end-user overruled some of these values by setting 59% for the *A-preference* of *prize* line and 50% for the *tax* line. It is important to note that no preferences are set over the attributes that are mapped to the *dbom:id* property.

In DBOM, the conclusion is represented as a SQL query where the set $\bar{x}$ correspond to elements, i.e. relation at-

tributes or functions applied to relation attributes, in the SELECT clause. This support of built-in SQL functions, e.g. string concatenation or arithmetical operations, enables to define one-to-one as well as n-to-1 mappings, i.e. mappings that make a correspondance between one (respectively n) elements of the source to one element of the target.

We now report concerning the efficiency and scalability of our Full-Preference algorithm in a practical mapping scenario. We have selected the domain of medical informatics and in particular drug related databases. The experiments were executed on a machine with an Intel Pentium D 3GHz with 2GB of memory and PostgreSQL 8.2. Each experiments were performed 5 times and we report the average values of these experiments. A first scenario (S1) involves a database (DB1) of 3.000 OTC (Over The Counter) drugs storing all the information concerning SPC information (Summary of Product Characteristics), i.e. drug name, composition, therapeutic class, posology, contra-indications, side-effects, etc.. This scenario only involves a single source while the 3 other scenarios map several sources. In the second scenario (S2), a new database (DB2) is introduced. It stores information about drugs related to the respiratory system. All 300 drugs of this database overlap with the set of tuples of DB1. Scenario S3 adds a homeopathy drug database of 400 products which overlap with DB1 and DB2. Finally scenario S4 introduces a database containing OTC drugs from a given pharmaceutical laboratory. Its set of tuples overlaps with DB1, DB2 and DB3.

The scenarios all aim to generate assertions in the ABox for the 3000 drug products. In all scenarios, Drug individuals are generated with datatype properties concerning 8 attributes. In scenario S1, all attributes are coming from DB1, while in other scenarios they are distributed over the set of sources and thus imply the machinery of the Full-Preference algorithm. The results of running the scenarios are represented in Figure 5 where 3 aspects are emphasized: view-level only involves R-preferences while the two attribute levels correspond to A-preferences settings. The figure highlights that in concrete cases, our algorithm is practical and scales well. By concrete cases, we mean that for a given (target) ontology mapped element, the number of views is not too large, so far we have not encountered situations with more than 5 views for a given ontology element. We have conducted concrete and practical experiments in the medicine related, biological and meteorological domains with many sources and also obtained positive results with this method.

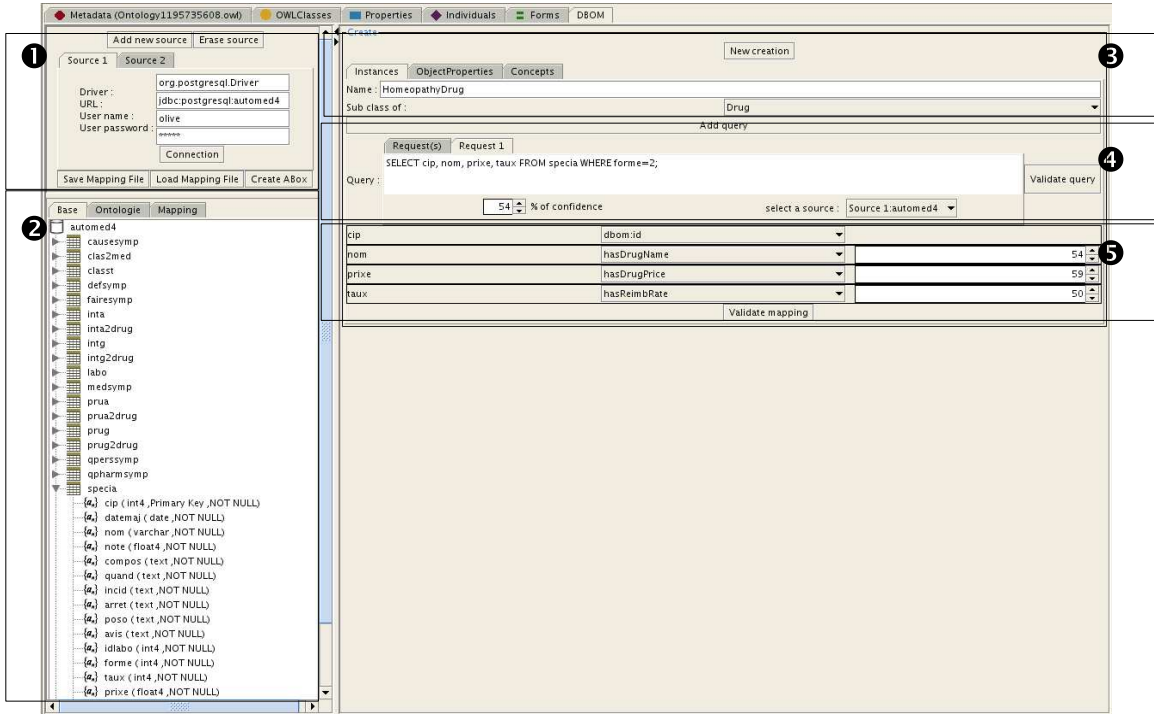


Figure 4: DBOM screenshot

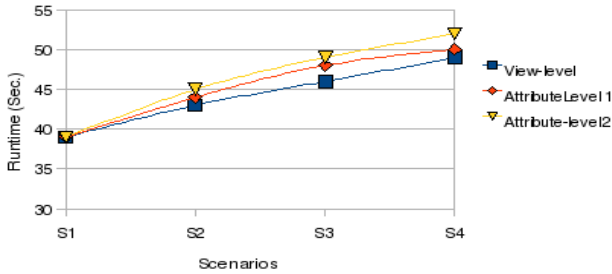


Figure 5: Evaluation of the Full-Preference algorithm

6. RELATED WORK

In this section, we present related work in the fields of (i) mapping technologies between structured and semi-structured databases and ontologies, (ii) using preferences in data integration. There are many solutions related to the design of expressive and computationally efficient mapping technologies between different types of databases and ontologies. The characteristics of DBOM make it similar to D2R MAP [3]. However important differences between these two solutions are: (i) in the terminological axiomatization possibilities of DBOM which enable the creation of ontologies as expressive as OWL DL, (ii) preference-based approach which enables to instantiate fine-grained knowledge bases and (iii) providing a GUI interface that enables to interact directly with all the functionalities of the Protégé editor, e.g.

adding restrictions to concepts created with the DBOM tab, checking for ontology consistency, checking to which OWL language the created ontology belongs to, querying the ontology with the SPARQL language, classification of concepts and properties, etc..

To the best of our knowledge, the first attempt to introduce parameters setting qualitative and quantitative descriptions of sources in a data integration system is [14]. In this paper, the authors characterize each source with two parameters : (i) soundness to qualify the confidence one can have on the answers proposed by the source, (ii) completeness to measure the amount of relevant data stored in the source. However, this approach can hardly be compared to ours since it does not constrain the target schema. In [8] a data integration system based on LAV mappings is enriched with information about source preferences. The meaning of preference assertions rests upon the fact that preference over the sources correspond to preferences between mapping assertions. This is due to the adoption of a LAV mapping which is not the case in DBOM. In [9], the authors formalize the declaration of preferences among the sources in a data integration system. As in DBOM, the mapping approach is GAV but the semantics differs from our solution and is based on repairing the data stored at the sources in the presence of global inconsistency. Using this approach, preferences expressed over the sources can easily be considered to solve inconsistencies. To the best of our knowledge, our solution is the first approach to consider preference setting over attributes of the views in a data integration setting.

7. CONCLUSION

In this paper, we presented a data integration solution that enables to create and instantiate knowledge bases com-

pliant with the Semantic Web from relational databases. The advantages of this solution are the following: (i) providing a simple, yet effective, mapping language that provides a solution to the impedance mismatch, (ii) enabling the instantiation of fine-grained consistent knowledge bases with the use of preferences on both the views and attribute of the views of the mapping and (iii) proposing a user-friendly GUI that is totally integrated in the Protégé ontology and knowledge editor.

In future works, we would like to consider a more expressive mapping approach, i.e. Global-Local-As-View (GLAV), and modify accordingly the mapping language in a such a way that data contained in XML documents can also be integrated.

8. REFERENCES

- [1] S. Abiteboul, R. Hull, and V. Vianu. *Foundations of Databases*. Addison Wesley, 1995.
- [2] F. Baader, D. Calvanese, D.L. McGuinness, D. Nardi, P.F. Patel-Schneider, *The Description Logic Handbook: Theory, Implementation, and Applications*, New York, USA: Cambridge University Press, 2003. Germany: Springer-Verlag, 1998.
- [3] C. Bizer, “D2R MAP - A Database to RDF Mapping Language” in *Proc.WWW’03*, 2003
- [4] O. Curé, “Semi-automatic data Migration in a Self-medication Knowledge-Based System” in *Proc.WM’05*, 2005, pp. 373-383.
- [5] O. Curé and R. Squelbut, “Data Integration Targeting a Drug Related Knowledge Base” in *Proc.EDBT Workshop’06*, 2006, pp. 411-422.
- [6] O. Curé, F. Jochaud, Preference-Based Integration of Relational Databases into a Description Logic. in *Proc.DEXA’07*, 2007, pp. 854-863.
- [7] Dean, M., Schreiber, G. : *OWL Web Ontology Language Reference*. 2004. W3C Recommendation 10 February 2004
- [8] G. De Giacomo, D. Lembo, M. Lenzerini, R. Rosati, “Tackling inconsistencies in data integration through source preferences” in *Proc.IQIS’04* 2004, pp.27-34.
- [9] G. Greco, D. Lembo, “Data integration with preferences among sources ” in *Proc.ER’04*, 2004, pp.231-244.
- [10] R. Fagin, P. Kolaitis, R. Miller, L. Popa, “Data exchange : semantics and query answering” in *Proc.ICDT’03*, 2003, pp. 207-224.
- [11] M. Lenzerini, “Data integration: a theoretical perspective” in *Proc.PODS’02*,2002, pp 233-246.
- [12] H. Levesque, G. Lakemeyer, *The Logic of Knowledge Bases*. Cambridge, USA: MIT Press, 2001.
- [13] A. Levy, “Obtaining Complete Answers from Incomplete Databases” in *Proc.VLDB’96*,1996, pp 402-412.
- [14] A. Mendelzon, G. Mihaila, “Querying partially sound an complete data sources” in *Proc.PODS’01*, 2001, pp. 162-170.
- [15] B. Motik, I. Horrocks, U. Sattler, “Bridging the Gap Between OWL and Relational Databases” in *Proc.WWW’07*, 2007, pp.807-816.
- [16] A. Schaerf, “Reasoning with individuals in concept languages” *Data and Knowledge Engineering*, vol. 13, number 2, pp. 141-176, 1994.
- [17] S.Tobies, “Complexity results and practical algorithms for logics in Knowledge Representation”. PhD Thesis, LuFG Theoretical Computer Science, RWTH-Aachen, Germany, 2001.